

Test Plan and Test Cases

Algorithmic Trading System

Document Control

Document Version:	Title:	Test	Plan	and	Test	Cases	for	Algorithmic	Trading	System	1.0
--------------------------	---------------	------	------	-----	------	-------	-----	-------------	---------	--------	-----

Date: April 24, 2025

Author: Senior Quality Assurance Engineer, Hedge Fund

Status: Draft

Distribution: Development Team, Product Owner, Trading Team, Compliance Team, Stakeholders

Revision History

Version	Date	Author	Description
1.0	04/24/2025	Senior QA Engineer	Initial draft of Test Plan and Test Cases

Approval

Role	Name	Signature	Date

Product Owner [To be assigned]

Head of Trading	[To be assigned]
-----------------	------------------

Compliance Officer [To be assigned]

QA Lead	[To be assigned]
---------	------------------

Table of Contents

- | | | | | |
|--------|------------------------------|----------------------|--------------------------|--|
| 1. | Introduction | | | |
| 1.1. | | | Purpose | |
| 1.2. | | | Scope | |
| 1.3. | | | Objectives | |
| 1.4. | Assumptions | | and | |
| 1.5. | Definitions and Acronyms | | Constraints | |
| 2. | Test | Plan | Overview | |
| 2.1. | | Testing | Objectives | |
| 2.2. | Scope | | of | |
| 2.3. | | Test | Testing | |
| 2.4. | | Test | Levels | |
| 2.5. | | Test | Types | |
| 2.6. | Roles | | Environment | |
| 2.7. | | Test | Responsibilities | |
| 2.8. | Entry | Test | Schedule | |
| 2.9. | | and | Exit | |
| 2.10. | | Test | Criteria | |
| | | | Deliverables | |
| 3. | Test | | Cases | |
| 3.1. | Unit | | Test | |
| 3.1.1. | Data | | Ingestion | |
| 3.1.2. | Data | | Processing | |
| 3.1.3. | Data | | Storage | |
| 3.1.4. | | Strategy | | |
| 3.1.5. | Order | | Execution | |
| 3.1.6. | Risk | | Management | |
| 3.1.7. | AI | | Agents | |
| 3.1.8. | | Reporting | | |
| 3.1.9. | Monitoring | and | Logging | |
| 3.2. | Integration | | Test | |
| 3.2.1. | Data | Ingestion | to | |
| 3.2.2. | Data | Processing | to | |
| 3.2.3. | Strategy | Engine | to | |
| 3.2.4. | Order | Execution | to | |

3.2.5.	AI	Agents	Framework	Integration
3.2.6.	API	Gateway	to	Dashboard
3.2.7.	Reporting	Module	to	Data Storage
3.3.	System		Test	Cases
3.3.1.	End-to-End		Trading	Workflow
3.3.2.	Market	Watch	Dashboard	Functionality
3.3.3.		Backtesting		Workflow
3.3.4.	Compliance	and	Audit	Logging
3.4.	Performance		Test	Cases
3.4.1.		Latency		Testing
3.4.2.		Throughput		Testing
3.4.3.		Scalability		Testing
3.5.	Security		Test	Cases
3.5.1.	Authentication		and	Authorization
3.5.2.		Data		Encryption
3.5.3.		Input		Validation
3.6.	Usability		Test	Cases
3.6.1.		Dashboard		Usability
3.6.2.		Accessibility		Testing
3.7.	Compliance		Test	Cases
3.7.1.		Regulatory		Compliance
3.7.2.	Audit Trail Integrity			
4.	Test	Execution	and	Reporting
4.1.	Test		Execution	Process
4.2.		Defect		Management
4.3.	Test Reporting			
5.	Missing	Information	and	Best Practices
5.1.		Missing		Information
5.2.	Best Practices Applied			
6.	Glossary			
7.	Appendices			
7.1.	Test	Test	Data	Samples
7.2.	Test	Tools	and	Frameworks
7.3.	References			

1. Introduction

1.1. Purpose

The Test Plan and Test Cases document for the Algorithmic Trading System (ATS) outlines the strategy, methodologies, and detailed test cases to ensure the system meets functional, non-functional, and regulatory requirements. This document serves as a comprehensive guide for the Quality Assurance (QA) team to validate the ATS's functionality, performance, security, usability, and compliance. It ensures that the system is robust, reliable, and ready for deployment in a simulated trading environment, aligning with the hedge fund's strategic objectives.

1.2. Scope

The ATS is a high-performance platform for algorithmic paper trading across stocks, indices, forex, and commodities on global exchanges (NYSE, NASDAQ, LSE, NSE). Integrated with the Interactive Brokers (IBKR) Paper Trading API, it uses Polars, DuckDB, and AI agents to support automated trading. The test scope includes:

- **Functional Testing:** Validate data ingestion, processing, storage, trading strategies, order execution, risk management, AI agents, dashboard, reporting, and monitoring.
- **Non-Functional Testing:** Ensure performance (<100ms latency), scalability (10 users, 1TB data), security (encryption, authentication), reliability (99.9% uptime), and usability (WCAG 2.1).
- **Compliance Testing:** Verify adherence to SEC, GDPR, MiFID II, and SEBI regulations.
- **Integration Testing:** Test interactions between components and external systems (IBKR API).
- **System Testing:** Validate end-to-end workflows, including trading and backtesting.
- **Test Environment:** Local deployment on Lenovo Legion Pro 5.

Out of Scope:

- Live trading functionality.
- Integration with brokers other than IBKR.
- Cryptocurrency or non-traditional asset support.
- Mobile app testing (dashboard is web-based).
- Advanced AI model training validation.

1.3. Objectives

- Ensure 100% coverage of functional requirements.
- Achieve >80% code coverage through unit and integration tests.
- Validate end-to-end latency <100ms and throughput of 10,000 updates/second.
- Confirm 99.9% uptime with fault-tolerant mechanisms.
- Verify compliance with SEC, GDPR, MiFID II, and SEBI regulations.
- Ensure dashboard usability with <5 minutes training time.
- Identify and mitigate security vulnerabilities.
- Validate scalability for 10 concurrent users and 1TB historical data.

1.4. Assumptions and Constraints

Assumptions:

- IBKR Paper Trading API provides reliable mock data and execution.
- Test environment mirrors production (Lenovo Legion Pro 5).
- QA team is proficient in Python, pytest, Locust, and OWASP ZAP.
- Traders provide feedback during usability testing.
- Budget supports 3-6 month testing phase.

Constraints:

- Timeline: 3-6 months, aligned with development.
- Hardware: Limited to Lenovo Legion Pro 5.
- IBKR API limits: 50 req/s (data), 100 msg/s (orders).
- Compliance: Must meet SEC, GDPR, MiFID II, SEBI standards.
- No cloud infrastructure; local deployment only.

1.5. Definitions and Acronyms

- **ATS:** Algorithmic Trading System.
- **IBKR:** Interactive Brokers.
- **OHLCV:** Open, High, Low, Close, Volume.
- **VW SMA:** Volume-Weighted Simple Moving Average.
- **MFI:** Money Flow Index.
- **ATR:** Average True Range.
- **Polars:** Data processing library.
- **DuckDB:** Analytical database.
- **pytest:** Python testing framework.
- **Locust:** Performance testing tool.
- **OWASP ZAP:** Security testing tool.

2. Test Plan Overview

2.1. Testing Objectives

- Validate functional correctness of all components.
- Ensure non-functional requirements (performance, security, usability) are met.
- Verify integration with IBKR API and internal services.
- Confirm regulatory compliance through audit trails.
- Achieve high test coverage and defect resolution before deployment.
- Validate system reliability under normal and stress conditions.

2.2. Scope of Testing

Components:

- Data Ingestion Service
- Data Processing Service
- Data Storage Service

- Strategy Engine
- Order Execution Module
- Risk Management Module
- AI Agents Framework
- Market Watch Dashboard
- Reporting Module
- Monitoring and Logging Service

Features:

- Real-time and historical market data processing.
- Trend-Following and Support and Resistance strategies.
- Order placement and execution in IBKR paper trading.
- Risk controls (position sizing, stop-loss, drawdown limits).
- AI-driven analysis and execution.
- Responsive dashboard with charts and trading controls.
- Performance, backtesting, and compliance reports.

2.3. Test Levels

- **Unit Testing:** Validate individual components (e.g., indicator calculations).
- **Integration Testing:** Test interactions between components and external systems.
- **System Testing:** Verify end-to-end workflows (e.g., trading, backtesting).
- **Acceptance Testing:** Confirm system meets user and regulatory requirements.

2.4. Test Types

- **Functional:** Validate features against requirements.
- **Non-Functional:**
 - Performance: Latency, throughput, scalability.
 - Security: Authentication, encryption, input validation.
 - Usability: Dashboard navigation, accessibility.
 - Reliability: Fault tolerance, uptime.
- **Compliance:** Audit trails, regulatory adherence.
- **Regression:** Ensure new changes do not break existing functionality.

2.5. Test Environment

- **Hardware:** Lenovo Legion Pro 5 (NVIDIA RTX 3060).
- **OS:** Ubuntu 22.04.
- **Software:**
 - Python 3.10, FastAPI, ib_insync, Polars, DuckDB.
 - React 18, Tailwind CSS, Plotly.js.
 - RabbitMQ, Redis, Prometheus, Grafana.
 - Docker Compose for service orchestration.
- **Network:** Localhost for internal services, external IBKR API connection.
- **Data:** Mock OHLCV, trade logs, and portfolio data from IBKR.

2.6. Roles and Responsibilities

Role	Responsibilities
QA Lead	Oversees testing, defines strategy, reviews results
QA Engineers	Write and execute test cases, report defects
Developers	Fix defects, support integration testing
Traders	Provide feedback during usability testing
Compliance Officer	Validate compliance-related test cases
IT Administrator	Set up test environment, manage infrastructure

2.7. Test Schedule

Phase	Duration	Activities
Test Planning	2 weeks	Define test plan, identify test cases
Test Case Design	4 weeks	Write detailed test cases, prepare test data
Unit Testing	4 weeks	Test individual components, achieve >80% coverage

Phase	Duration	Activities
Integration Testing	6 weeks	Test component interactions, IBKR integration
System Testing	4 weeks	Validate end-to-end workflows, backtesting
Performance Testing	3 weeks	Measure latency, throughput, scalability
Security Testing	2 weeks	Test authentication, encryption, vulnerabilities
Usability Testing	2 weeks	Validate dashboard with traders
Compliance Testing	2 weeks	Verify audit trails, regulatory adherence
Regression Testing	2 weeks	Ensure no regressions after defect fixes
Acceptance Testing	1 week	Final validation with stakeholders

Total Duration: ~26 weeks (6 months), overlapping with development.

2.8. Entry and Exit Criteria

Entry Criteria:

- Requirements finalized and approved.
- Test environment set up and stable.
- Test cases documented and reviewed.
- Mock data prepared for IBKR API.
- Development code frozen for testing phase.

Exit Criteria:

- 100% functional test cases passed.
- 80% code coverage achieved.
- All critical/high-severity defects resolved.
- Performance targets met (<100ms latency, 10,000 updates/s).
- Security vulnerabilities mitigated.
- Compliance requirements validated.
- Stakeholder approval for deployment.

2.9. Test Deliverables

- Test Plan and Test Cases document.
- Test data sets (OHLCV, trade logs, portfolio).
- Test scripts (pytest, Locust).
- Test execution reports.
- Defect reports with severity and status.
- Code coverage reports.
- Performance and security test results.
- Usability feedback from traders.
- Compliance validation report.

2.10. Risks and Mitigation

Risk	Likelihood	Impact	Mitigation
IBKR API downtime	Medium	High	Use mock data, cache historical data
Insufficient test data	Low	Medium	Generate synthetic OHLCV, use IBKR historical
Performance bottlenecks	Medium	High	Optimize Polars/DuckDB, profile with Locust
Security vulnerabilities	Medium	High	Regular OWASP ZAP scans, encrypt data
Compliance gaps	Low	High	Involve compliance officer early
Trader feedback delays	Medium	Medium	Schedule usability tests in advance

3. Test Cases

3.1. Unit Test Cases

3.1.1. Data Ingestion Service

Objective: Validate data fetching from IBKR API.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-ING-001	Fetch real-time OHLCV	IBKR connected	API Ticker: "AAPL", Timeframe: "1m"	OHLCV JSON with valid data	Data matches IBKR response
TC-ING-002	Fetch historical OHLCV	IBKR connected	API Ticker: "AAPL", Timeframe: "1d", Start: "2025-01-01"	Array of OHLCV JSON	Data matches historical records
TC-ING-003	Handle invalid ticker	IBKR connected	API Ticker: "INVALID"	Error: "invalid_ticker"	Error logged, no crash
TC-ING-004	Respect rate limit	IBKR connected	API 51 requests/s	Error: "rate_limit_exceeded"	Retry after 1s
TC-ING-005	Cache data in Redis	Redis running	Ticker: "AAPL", Timeframe: "1m"	Data stored with 60s TTL	Redis key exists
TC-ING-006	Reconnect on disconnect	IBKR disconnected	API Simulate disconnect	Reconnect within 5s	Connection restored

3.1.2. Data Processing Service

Objective: Validate data normalization and indicator calculations.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PROC-001	Normalize timestamps	Raw OHLCV in Redis	OHLCV with mixed timezones	with UTC timestamps	All timestamps in UTC
TC-PROC-002	Clean outliers	Raw OHLCV	OHLCV price $> 3\sigma$	with Outliers removed	Prices within 3σ
TC-PROC-003	Interpolate missing values	Raw OHLCV	OHLCV missing close	with Linear interpolation	No missing values
TC-PROC-004	Calculate VW SMA	Clean OHLCV	OHLCV 55 periods	for VW value	SMA Matches formula: $\Sigma(\text{Price} \times \text{Volume}) / \Sigma \text{Volume}$
TC-PROC-005	Calculate MFI	Clean OHLCV	OHLCV 14 periods	for MFI value	Matches formula: $100 - (100 / (1 + \text{Positive/Negative Money Flow}))$
TC-PROC-006	Calculate ATR	Clean OHLCV	OHLCV 21 periods	for ATR value	Matches VW SMA of True Range
TC-PROC-007	Handle corrupted data	Raw OHLCV	OHLCV negative volume	with Skip record, log error	No crash, error logged

3.1.3. Data Storage Service

Objective: Validate data storage and retrieval.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-STOR-001	Store OHLCV	DuckDB running	OHLCV JSON	Data in ohlcv table	Query returns inserted data
TC-STOR-002	Store trade log	DuckDB running	Trade JSON	Data in trade_log table	Query returns trade
TC-STOR-003	Store portfolio	DuckDB running	Portfolio JSON	Data in portfolio table	Query returns position
TC-STOR-004	Partition date/ticker	by DuckDB running	OHLCV for multiple dates	Partitioned Parquet files	Files exist in partitions
TC-STOR-005	Query performance	Data stored	SELECT * WHERE ticker="AAPL"	Results <100ms	in Query <100ms time
TC-STOR-006	Handle write failure	Disk full	OHLCV JSON	Retry 3 times, log error	No crash, error logged

3.1.4. Strategy Engine

Objective: Validate trading strategy logic.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-STRAT-001	Trend-Following signal	buy Processed OHLCV	MFI(14) > 34-day VW SMA, price > 55-day VW SMA	Buy signal	Signal "BUY" action:
TC-STRAT-002	Trend-Following signal	sell Processed OHLCV	MFI(14) < 21-day VW SMA	Sell signal	Signal "SELL" action:
TC-STRAT-003	Support/Resistance buy signal	Processed OHLCV	Price breaks R1, high volume	Buy signal	Signal "BUY" action:
TC-STRAT-004	Support/Resistance sell signal	Processed OHLCV	Price breaks S1	Sell signal	Signal "SELL" action:
TC-STRAT-005	Position sizing	Account balance: £10,000	Risk: 1%, ATR: 2.0	Quantity calculated	Matches formula: $(0.01 \times 10,000) / (2 \times \text{ATR})$
TC-STRAT-006	Handle invalid data	Corrupted OHLCV	Missing price	Skip signal, log error	No crash, error logged

3.1.5. Order Execution Module

Objective: Validate order placement and monitoring.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-ORD-001	Place market order	IBKR connected	Buy AAPL, 100 shares	Order "PENDING"	status: Order returned ID
TC-ORD-002	Place limit order	limit IBKR connected	Buy AAPL, 100 shares, price: 150.90	Order "PENDING"	status: Order returned ID

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-ORD-003	Place order	IBKR connected	Buy AAPL, stop-loss, take-profit	Order status: "PENDING"	Order ID returned
TC-ORD-004	Monitor order status	Order placed	Order ID	Status: "FILLED"	Status updated in DuckDB
TC-ORD-005	Handle order	IBKR connected	Negative quantity	Error: "invalid_quantity"	No order placed
TC-ORD-006	Respect limit	IBKR connected	101 msg/s	Error: "rate_limit_exceeded"	Retry after 1s

3.1.6. Risk Management Module

Objective: Validate risk controls.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-RISK-001	Validate position size	Account balance: £10,000	Trade with 1% risk, ATR: 2.0	Approved trade	Size matches formula
TC-RISK-002	Reject high-risk trade	Account balance: £10,000	Trade with 3% risk	Error: "risk_violation"	Trade rejected
TC-RISK-003	Enforce stop-loss	Trade signal	Buy AAPL, stop-loss: $2 \times \text{ATR}$	Stop-loss set	Stop-loss price valid
TC-RISK-004	Trigger circuit breaker	Market data	Price change >5% in 5min	Trading halted	Alert logged
TC-RISK-005	Limit portfolio risk	Portfolio data	Open risk >10%	Error: "max_open_risk"	Trade rejected
TC-RISK-006	Handle drawdown	Portfolio data	Drawdown >20%	Risk reduced to 0.5%	Risk parameters updated

3.1.7. AI Agents Framework

Objective: Validate AI-driven tasks.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-AI-001	Trade execution timing	Market data	OHLCV, volatility	Optimized trade price	Price within 0.1% of expected
TC-AI-002	Risk adjustment	Portfolio data	High exposure	Reduced position size	Size within risk limits
TC-AI-003	Market prediction	Historical OHLCV	14-day data	Uptrend prediction	Matches LSTM model output
TC-AI-004	Handle API limits	API rate	IBKR connected 51 req/s	Throttled requests	No API errors
TC-AI-005	Log explanations	SHAP	Trade decision	Buy signal	SHAP values logged, Explanations queryable

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-AI-006	Fallback on failure	Agent failure	Invalid model output	Manual mode	No crash, alert logged

3.1.8. Reporting Module

Objective: Validate report generation.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-REP-001	Trade performance report	Trade data	Account ID, 2025-01-01 to 2025-04-24	P&L, Sharpe ratio	Matches calculated values
TC-REP-002	Portfolio analytics	Portfolio data	Account ID	Exposure, VaR	Matches pandas calculations
TC-REP-003	Backtesting report	Historical data	Strategy ID, ticker	Total return, drawdown	Matches backtest results
TC-REP-004	Compliance report	Trade logs	Account ID	Transaction logs	Immutable, queryable
TC-REP-005	Export to CSV	Report data	Format: "csv"	CSV file	File downloadable
TC-REP-006	Handle query failure	DuckDB down	Report query	Retry 3 times, log error	No crash, error logged

3.1.9. Monitoring and Logging Service

Objective: Validate metrics and logs.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-MON-001	Collect metrics	Prometheus running	API call	Latency <100ms	Metric in Prometheus
TC-MON-002	Visualize in Grafana	Grafana running	Metrics	Dashboard with latency	Graph displayed
TC-MON-003	Log API call	API request	GET /market-data	JSON log	Log in DuckDB
TC-MON-004	Alert on high latency	Prometheus running	Latency >100ms	Email/SMS alert	Alert received
TC-MON-005	Handle logging failure	DuckDB down	Log entry	Retry 3 times	No crash, error logged
TC-MON-006	Query historical logs	Logs stored	SELECT * WHERE level="ERROR"	Error logs	Query returns results

3.2. Integration Test Cases

3.2.1. Data Ingestion to Data Processing

Objective: Validate data flow from ingestion to processing.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-001	Real-time data flow	IBKR, running	Redis AAPL OHLCV	Processed in DuckDB	OHLCV Data processed within 100ms

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-002	Historical data flow	IBKR, running	Redis AAPL historical data	Processed OHLCV in DuckDB	All records processed
TC-INT-003	Error propagation	IBKR error	returns Invalid ticker	Error logged, crash	no Processing invalid data skips

3.2.2. Data Processing to Strategy Engine

Objective: Validate indicator data to signal generation.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-004	Trend-Following signal	Processed OHLCV	MFI > VW SMA	Buy signal via RabbitMQ	Signal received
TC-INT-005	Support/Resistance signal	Processed OHLCV	Price breaks R1	Buy signal via RabbitMQ	Signal received
TC-INT-006	Invalid data handling	Corrupted OHLCV	Missing price	No signal, error logged	No crash

3.2.3. Strategy Engine to Order Execution

Objective: Validate signal to order placement.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-007	Execute buy signal	IBKR connected	Buy signal for AAPL	Order placed	Order returned ID
TC-INT-008	Execute sell signal	IBKR connected	Sell signal for AAPL	Order placed	Order returned ID
TC-INT-009	Handle invalid signal	Invalid signal	Negative quantity	Error logged	No order placed

3.2.4. Order Execution to Risk Management

Objective: Validate risk checks before execution.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-010	Approve trade	valid Risk set parameters	1% risk trade	Order approved	Order placed
TC-INT-011	Reject trade	high-risk Risk set parameters	3% risk trade	Error: "risk_violation"	Order rejected
TC-INT-012	Trigger breaker	circuit Market data	Price change >5%	Trading halted	Alert logged

3.2.5. AI Agents Framework Integration

Objective: Validate AI agent interactions.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-013	AI trade recommendation	Market data	OHLCV, volatility	Buy recommendation	Matches model output
TC-INT-014	AI risk adjustment	Portfolio data	High exposure	Reduced position	Within risk limits
TC-INT-015	Human override	AI recommendation	Override command	Manual execution	AI paused

3.2.6. API Gateway to Dashboard

Objective: Validate API to dashboard communication.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-016	Real-time data display	API, dashboard running	AAPL OHLCV	Data in dashboard	Updates every 5-10s
TC-INT-017	Order placement	API, dashboard running	Buy AAPL	Order status displayed	Status matches API
TC-INT-018	Handle failure	API API down	GET /market-data	Error message	Dashboard graceful degradation

3.2.7. Reporting Module to Data Storage

Objective: Validate report generation from storage.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-INT-019	Trade performance report	Trade data in DuckDB	Account ID	Report generated	Matches trade data
TC-INT-020	Compliance report	Trade logs	Account ID	Immutable logs	Queryable logs
TC-INT-021	Handle storage failure	DuckDB down	Report query	Retry, log error	No crash

3.3. System Test Cases

3.3.1. End-to-End Trading Workflow

Objective: Validate complete trading process.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SYS-001	Execute buy trade	System running	AAPL signal buy	Order filled, portfolio updated	Trade logged, P&L updated
TC-SYS-002	Execute sell trade	System running	AAPL signal sell	Order filled, portfolio updated	Trade logged, P&L updated
TC-SYS-003	Handle violation	risk System running	High-risk trade	Trade rejected	Error logged, no trade
TC-SYS-004	Circuit breaker activation	System running	Price change >5%	Trading halted	Alert logged

3.3.2. Market Watch Dashboard Functionality

Objective: Validate dashboard features.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SYS-005	Display real-time data	Dashboard running	AAPL OHLCV	Data in table	Updates every 5-10s
TC-SYS-006	Render candlestick chart	Dashboard running	AAPL OHLCV	Chart with SMA, MFI	VW Chart renders in <1s
TC-SYS-007	Place order via dashboard	Dashboard running	Buy AAPL, 100 shares	Order status displayed	Order ID matches API
TC-SYS-008	Manage watchlist	Dashboard running	Add AAPL	AAPL in watchlist	Watchlist updated

3.3.3. Backtesting Workflow

Objective: Validate strategy backtesting.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SYS-009	Backtest Following	Trend- Historical data	AAPL, 2020-2025	Report with returns, Sharpe	Matches expected metrics

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SYS-010	Backtest Support/Resistance	Historical data	AAPL, 2020-2025	Report drawdown	with Matches expected metrics
TC-SYS-011	Handle missing data	Historical data	Partial OHLCV	Error logged, partial results	No crash

3.3.4. Compliance and Audit Logging

Objective: Validate regulatory compliance.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SYS-012	Log trade activity	System running	Buy AAPL	Immutable log	Log queryable, timestamped
TC-SYS-013	Retain logs for 7 years	Logs stored	Trade logs	Logs accessible	Logs not deleted
TC-SYS-014	Detect market abuse	Trade data	Spoofing pattern	Alert logged	Compliance officer notified

3.4. Performance Test Cases

3.4.1. Latency Testing

Objective: Validate system latency.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PERF-001	End-to-end latency	System running	AAPL signal	buy Latency <100ms	Measured via Prometheus
TC-PERF-002	Indicator calculation	OHLCV data	100 scrips	<1s	Measured via Locust
TC-PERF-003	Dashboard refresh	Dashboard running	AAPL OHLCV	Every 5-10s	No lag observed

3.4.2. Throughput Testing

Objective: Validate system throughput.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PERF-004	Data updates	System running	10,000 updates/s	All processed	No dropped updates
TC-PERF-005	Trade execution	System running	1,000 trades/min	All executed	No failures
TC-PERF-006	Report generation	Data stored	1 year of data	<10s	Report generated

3.4.3. Scalability Testing

Objective: Validate system scalability.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-PERF-007	Concurrent users	System running	10 users	No performance degradation	Latency <100ms
TC-PERF-008	Historical data	Data stored	1TB data	Queries <100ms	No timeout
TC-PERF-009	High load	System running	100 scrips, 10 users	Stable performance	No crashes

3.5. Security Test Cases

3.5.1. Authentication and Authorization

Objective: Validate secure access.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SEC-001	Valid token	JWT Keycloak running	Valid token	Access granted	API call succeeds
TC-SEC-002	Invalid token	Keycloak running	Invalid token	401 Unauthorized	Access denied
TC-SEC-003	MFA for orders	Keycloak running	Order without MFA	401 Unauthorized	Access denied
TC-SEC-004	Role-based access	Keycloak running	Trader accesses admin endpoint	403 Forbidden	Access denied

3.5.2. Data Encryption

Objective: Validate data security.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SEC-005	TLS 1.3 for API	Nginx running	API call	Encrypted traffic	Wireshark shows TLS 1.3
TC-SEC-006	AES-256 storage	for DuckDB running	OHLCV data	Encrypted Parquet	Data unreadable without key
TC-SEC-007	Credential encryption	Vault running	API credentials	Encrypted vault	in No plaintext exposure

3.5.3. Input Validation

Objective: Validate input handling.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-SEC-008	Prevent injection	SQL API running	Malicious query	400 Bad Request	No database compromise
TC-SEC-009	Prevent XSS	Dashboard running	Script in input	Sanitized output	No script execution
TC-SEC-010	Validate quantity	order API running	Negative quantity	422 Unprocessable Entity	Order rejected

3.6. Usability Test Cases

3.6.1. Dashboard Usability

Objective: Validate user experience.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-USE-001	Learn navigation	Dashboard running	New user	<5min navigate	to User completes tasks
TC-USE-002	Place order	Dashboard running	Buy AAPL	Order placed	Order confirmed in <10s
TC-USE-003	View chart	Dashboard running	AAPL OHLCV	Chart renders	Chart visible in <1s

3.6.2. Accessibility Testing

Objective: Validate WCAG 2.1 compliance.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-USE-004	Keyboard navigation	Dashboard running	Navigate keyboard	via All elements accessible	No stuck focus
TC-USE-005	High contrast mode	Dashboard running	Enable contrast	high Readable text	Contrast ratio >4.5:1

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-USE-006	Screen reader support	Dashboard running	Use NVDA	ARIA labels read	All elements described

3.7. Compliance Test Cases

3.7.1. Regulatory Compliance

Objective: Validate regulatory adherence.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-COMP-001	Log all trades	System running	Buy AAPL	Immutable log	Log in DuckDB
TC-COMP-002	Retain logs for 7 years	Logs stored	Trade logs	Logs accessible	No deletion
TC-COMP-003	Monitor market abuse	Trade data	Spoofing pattern	Alert logged	Compliance notified

3.7.2. Audit Trail Integrity

Objective: Validate audit trail immutability.

Test Case ID	Description	Precondition	Input	Expected Output	Pass/Fail Criteria
TC-COMP-004	Prevent log modification	Logs stored	Attempt to edit log	Error: "immutable"	Log unchanged
TC-COMP-005	Query audit trail	Logs stored	SELECT * WHERE user_id="user123"	All user actions	Logs returned
TC-COMP-006	Log API access	API running	GET /market-data	Access logged	Log in DuckDB

4. Test Execution and Reporting

4.1. Test Execution Process

1. **Setup:** Configure test environment, load test data.
2. **Execution:**
 - o Run unit tests during development.
 - o Execute integration tests after component completion.
 - o Perform system, performance, security, and usability tests in staging.
 - o Conduct regression tests after defect fixes.
3. **Logging:** Record test results in JIRA with pass/fail status.
4. **Defect Reporting:** Log defects in JIRA with severity and steps to reproduce.
5. **Review:** QA Lead reviews results, escalates critical issues.

4.2. Defect Management

- **Severity Levels:**
 - o Critical: System crash, data loss, security breach.
 - o High: Major functionality failure, compliance violation.
 - o Medium: Minor functionality issue, usability flaw.
 - o Low: Cosmetic issues.
- **Process:**
 - o Log in JIRA with description, severity, and screenshots.
 - o Assign to developers for resolution.
 - o Retest after fix, update status.
 - o Escalate unresolved critical defects to stakeholders.
- **Resolution Time:**
 - o Critical: <24 hours.

- High: <48 hours.
- Medium: <1 week.
- Low: <2 weeks.

4.3. Test Reporting

- **Daily Reports:** Test execution status, defects found/resolved.
- **Weekly Reports:** Progress against schedule, coverage metrics.
- **Final Report:** Summary of test results, defects, and readiness for deployment.
- **Tools:** JIRA for tracking, pytest for coverage, Grafana for performance metrics.

5. Missing Information and Best Practices

5.1. Missing Information

- Exact IBKR API mock data specifications.
- Detailed compliance requirements for paper trading.
- Specific trader personas for usability testing.
- Performance benchmarks for Lenovo Legion Pro 5.
- News feed integration details.

5.2. Best Practices Applied

- **Mock Data:** Used synthetic OHLCV based on typical market patterns. **Reason:** Ensures comprehensive test coverage without live data.
- **Compliance:** Implemented 7-year log retention and immutable audit trails. **Reason:** Aligns with SEC/GDPR standards.
- **Usability:** Assumed intermediate traders for testing. **Reason:** Matches hedge fund user base.
- **Performance:** Targeted <100ms latency based on high-frequency trading standards. **Reason:** Ensures competitiveness.
- **Security:** Adopted OWASP ZAP and TLS 1.3. **Reason:** Industry-standard for secure systems.

6. Glossary

- **ATS:** Algorithmic Trading System.
- **IBKR:** Interactive Brokers.
- **OHLCV:** Open, High, Low, Close, Volume.
- **VW SMA:** Volume-Weighted Simple Moving Average.
- **MFI:** Money Flow Index.
- **ATR:** Average True Range.
- **pytest:** Python testing framework.
- **Locust:** Performance testing tool.

7. Appendices

7.1. Test Data Samples

OHLCV:

```
{
  "ticker": "AAPL",
  "exchange": "NASDAQ",
  "timestamp": "2025-01-01T10:00:00Z",
  "open": 150.25,
  "high": 151.10,
  "low": 149.80,
  "close": 150.90,
  "volume": 1000000
}
```

Trade Log:

```
{
  "trade_id": "T12345",
  "timestamp": "2025-01-01T10:00:00Z",
  "ticker": "AAPL",
  "strategy": "trend_following",
}
```

```
"action": "BUY",  
"quantity": 100,  
"price": 150.90,  
"status": "FILLED"  
}
```

7.2. Test Tools and Frameworks

- **pytest**: Unit and integration testing.
- **Locust**: Performance and load testing.
- **OWASP ZAP**: Security testing.
- **Postman**: API testing.
- **NVDA**: Accessibility testing.
- **Prometheus/Grafana**: Performance monitoring.
- **JIRA**: Defect tracking and reporting.

7.3. References

- IBKR API: <https://interactivebrokers.github.io/tws-api/>
 - Polars: <https://pola.rs/>
 - DuckDB: <https://duckdb.org/>
 - pytest: <https://docs.pytest.org/>
 - OWASP ZAP: <https://owasp.org/www-project-zap/>
-